

LAPORAN UJIAN TENGAH SEMESTER GENAP T.A 2025/2026

SISTEM INVENTARIS TOKO / GUDANG

Dosen Pengampu: Muhammad Panji Muslim, S.Pd., M.Kom.



MATA KULIAH:

PEMBANGUNAN PERANGKAT LUNAK BERORIENTASI SERVICE (SE.2)

Disusun Oleh:

Melva Fereyzha Kirana Myko Putri

2410511165

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" JAKARTA

2026

A. Pendahuluan

1. Latar Belakang

Pengelolaan inventaris toko dan gudang secara konvensional masih banyak dilakukan secara manual menggunakan catatan fisik atau spreadsheet sederhana. Proses ini tidak efisien karena rentan terhadap kesalahan pencatatan, sulit dilacak secara *real-time*, dan tidak transparan bagi seluruh pihak yang terlibat dalam rantai distribusi barang.

Perkembangan teknologi informasi mendorong kebutuhan akan sistem yang lebih terstruktur, *scalable*, dan mudah dikembangkan. Arsitektur *Service-Oriented Architecture* (SOA) khususnya *microservice* hadir sebagai solusi yang memungkinkan sistem dibangun dari komponen-komponen kecil yang independen, masing-masing bertanggung jawab atas satu domain bisnis tertentu.

Oleh karena itu, dibangunlah Sistem Inventaris Toko/Gudang berbasis *microservice* yang memungkinkan pengelolaan barang, stok, *supplier*, dan lokasi dilakukan secara digital dengan alur yang terstruktur, aman, dan dapat dipantau secara *real-time*.

2. Tujuan

Tujuan Pembangunan sistem ini adalah:

- a) Membangun sistem inventaris berbasis *microservice* dengan minimal 3 *service* independen yang berkomunikasi melalui REST API.
- b) Mengimplementasikan autentikasi menggunakan JWT dengan mekanisme *access token*, *refresh token*, dan *token blacklist*.
- c) Mengimplementasikan login alternatif menggunakan GitHub OAuth 2.0 dengan *Authorization Code Flow*.
- d) Membangun service inventaris menggunakan PHP CodeIgniter 4 dengan pola MVC yang dikonsumsi oleh service lain.
- e) Mengimplementasikan API Gateway sebagai *single entry point* dengan fitur JWT *validation* dan *rate limiting*.
- f) Memastikan setiap service memiliki database yang terpisah sesuai prinsip *microservice*.

3. Lingkup Sistem

Sistem Inventaris Toko/Gudang yang dibangun mencakup lingkup sebagai berikut:

- a) Manajemen Autentikasi: registrasi user, login dengan email/password, login dengan GitHub OAuth 2.0, refresh token, logout dengan blacklist token, dan pengelolaan profil user.
- b) Manajemen Barang (Items): penambahan, penampilan, pembaruan, dan penghapusan data barang beserta relasi ke *supplier*.
- c) Manajemen *Supplier*: pengelolaan data *supplier*/pemasok barang yang menyuplai item ke toko/gudang.
- d) Manajemen Lokasi: pengelolaan data lokasi gudang atau toko tempat barang disimpan.
- e) Manajemen Pergerakan Stok: pencatatan barang masuk (in) dan barang keluar (out) per lokasi beserta keterangan.
- f) Laporan Stok: laporan *summary* pergerakan stok yang mengagregas data dari *inventory service* secara *inter-service*.
- g) API Gateway: *routing*, validasi JWT, dan *rate limiting* untuk seluruh service.

B. Arsitektur Sistem

Sistem ini dibangun menggunakan arsitektur *microservice* yang terdiri dari 3 service independen dan 1 API Gateway sebagai *single entry point*. Setiap service memiliki tanggung jawab yang terpisah dan berkomunikasi melalui REST API.

Path	Service Tujuan	Port	Teknologi
/auth/	auth-service	3001	Node.js
/inventory/	inventory-service	8080	PHP CodeIgniter 4
/report/	report-service	3003	Node.js

Seluruh request dari klien hanya boleh masuk melalui API Gateway di port 8000. Gateway melakukan validasi JWT dan *rate limiting* sebelum meneruskan *request* ke *service* tujuan.

C. Justifikasi Pemisahan Service

Pada pendekatan monolitik, seluruh fitur autentikasi, inventaris, dan laporan berada dalam satu aplikasi dengan satu database. Pendekatan ini memiliki beberapa kelemahan:

- a) Jika satu modul mengalami error, seluruh sistem ikut terganggu.
- b) Scaling harus dilakukan untuk seluruh aplikasi meskipun hanya satu modul yang membutuhkan *resource* lebih.
- c) Perubahan kecil pada satu fitur mengharuskan *deployment* ulang seluruh aplikasi.
- d) Tim dengan keahlian berbeda tidak bisa bekerja secara paralel dan independen.

Dengan pendekatan *microservice* yang diterapkan pada sistem ini, setiap service dapat dikembangkan, dideploy, dan di-*scale* secara independen tanpa mempengaruhi service lain.

Justifikasi *auth-service* dipisah karena autentikasi adalah *cross-cutting concern* yang digunakan oleh seluruh service lain. Dengan memisahkan *auth-service*, logika JWT dan OAuth tidak tercampur dengan logika bisnis inventaris. Jika di masa depan ada service baru yang ditambahkan, service tersebut cukup berkomunikasi dengan *auth service* tanpa perlu mengimplementasikan ulang logika autentikasi.

Justifikasi *inventory-service* dipisah karena inventaris merupakan domain bisnis utama sistem ini dengan logika dan data yang kompleks mencakup manajemen barang, *supplier*, lokasi dan pergerakan stok. Memisahkannya memungkinkan pengembangan menggunakan *framework* CodeIgniter 4 secara independen tanpa harus menyesuaikan dengan teknologi service lain. Database inventory juga terpisah sehingga perubahan skema tidak memengaruhi service lain.

Justifikasi *report-service* dipisah karena pelaporan memiliki logika tersendiri yaitu mengambil dan mengagregas data pergerakan stok dari *inventory service* untuk menghasilkan laporan *summary*. Dengan memisahkan *report service*, alur bisnis pelaporan dapat dikembangkan secara independen. Service ini juga mendemonstrasikan komunikasi *inter-service* dengan mengkonsumsi *inventory service* melalui HTTP request menggunakan axios, sesuai dengan prinsip *microservice*.

D. Penjelasan Service

1. *Auth-Service (Port 3001)*

Auth service bertanggung jawab atas seluruh proses autentikasi dan otorisasi. Dibangun menggunakan Node.js dan Express dengan implementasi JWT dan GitHub OAuth 2.0. Fitur yang diimplementasikan:

- a) Registrasi user baru dengan enkripsi password menggunakan *bcryptjs*.
- b) Login menggunakan email dan password yang menghasilkan access token dan refresh token.
- c) Access token memiliki masa berlaku 15 menit, refresh token 7 hari.
- d) Endpoint refresh token untuk memperbarui access token yang expired.
- e) Endpoint logout dengan mekanisme token blacklist sehingga token yang sudah logout tidak bisa digunakan kembali.
- f) Login alternatif menggunakan GitHub OAuth 2.0 dengan Authorization Code Flow.
- g) Data user dari GitHub (nama, email, foto profil) disimpan ke database lokal dengan flag *oauth_provider*.

2. *Inventory-Service (Port 8080)*

Inventory service dibangun menggunakan PHP dengan *framework* CodeIgniter 4 mengikuti pola MVC. Pemisahan layer dilakukan secara jelas:

- a) Model (ItemModel.php, SupplierModel.php, LocationModel.php, StockMovementModel.php): menangani *query* database dan validasi data.
- b) Controller (ItemController.php, SupplierController.php, LocationController.php, StockMovementController.php): menangani logika bisnis dan *response*.
- c) Routes (Routes.php): mendefinisikan *endpoint* dan *mapping* ke *controller*.

Fitur yang diimplementasikan:

- a) CRUD barang (items) beserta relasi ke *supplier*.
- b) CRUD supplier yang menyediakan barang.
- c) CRUD lokasi gudang/toko.
- d) Pencatatan pergerakan stok masuk (in) dan keluar (out) per lokasi.
- e) Validasi input pada layer controller menggunakan *validation rules* CodeIgniter.

- f) *Paging* menggunakan parameter *page* dan *per_page* pada *endpoint listing*.
- g) *Filtering* berdasarkan nama item dan tipe pergerakan stok.
- h) *HTTP method* yang semantik (GET, POST, PUT, DELETE) dengan *response code* yang sesuai.

Database *inventory_db* memiliki 4 tabel ter-relasi: *suppliers*, *locations*, *items*, dan *stock_movements*.

3. Report-Service (Port 3003)

Report service dibangun menggunakan Node.js dan Express. *Service* ini mengkonsumsi *inventory service* melalui HTTP request menggunakan *library Axios*, mendemonstrasikan komunikasi *inter-service*. Fitur yang diimplementasikan:

- a) Mengambil data pergerakan stok dari *inventory service*.
- b) Menghitung *summary total barang masuk (in) dan keluar (out)*.
- c) Menampilkan total seluruh pergerakan stok beserta detailnya.

4. API Gateway (Port 8000)

API Gateway berfungsi sebagai *single entry point* untuk seluruh request dari klien. Dibangun menggunakan Node.js dan Express. Fitur yang diimplementasikan:

- a) *Routing request* ke service tujuan berdasarkan *path*.
- b) Validasi JWT di sisi *gateway* sebelum request diteruskan ke *service inventory* dan *report*.
- c) *Rate limiting* sebesar 60 request per menit per IP menggunakan *express-rate-limit*.
- d) Error handling apabila service tujuan tidak tersedia.

E. Database

Setiap service memiliki database yang terpisah sesuai prinsip *microservice*. Tidak ada tabel yang digunakan bersama antar service.

- a) Auth Service — *data/users.json* (file-based): menyimpan data pengguna termasuk informasi OAuth (*oauth_provider*, *avatar*) untuk mendukung login via GitHub.
- b) Inventory Service — *inventory_db*: terdiri dari 4 tabel ter-relasi:
 - i. *suppliers*: menyimpan data *supplier/pemasok barang*.

- ii. *locations*: menyimpan data lokasi gudang atau toko.
- iii. *items*: menyimpan data barang dengan relasi ke supplier.
- iv. *stock_movements*: menyimpan riwayat pergerakan stok masuk/keluar dengan relasi ke items dan *locations*.

F. Implementasi Keamanan

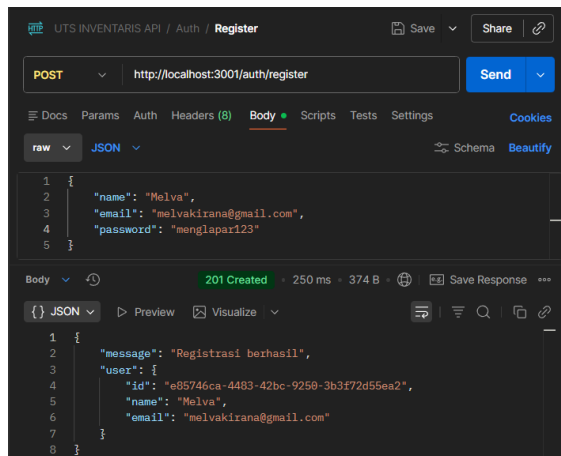
- a) JWT *access token* dengan masa berlaku 15 menit mencegah penyalahgunaan token yang bocor.
- b) *Refresh token* 7 hari memungkinkan user tetap login tanpa harus input password berulang.
- c) Token blacklist saat logout memastikan token yang sudah tidak digunakan tidak bisa dipakai kembali.
- d) GitHub OAuth 2.0 menggunakan Authorization Code Flow (bukan Implicit Flow yang sudah deprecated).
- e) JWT *validation* dilakukan di gateway sebelum request diteruskan ke service, bukan per-endpoint secara manual.
- f) *Rate limiting* 60 request per menit per IP mencegah abuse dan serangan *brute force*.

G. Kesimpulan

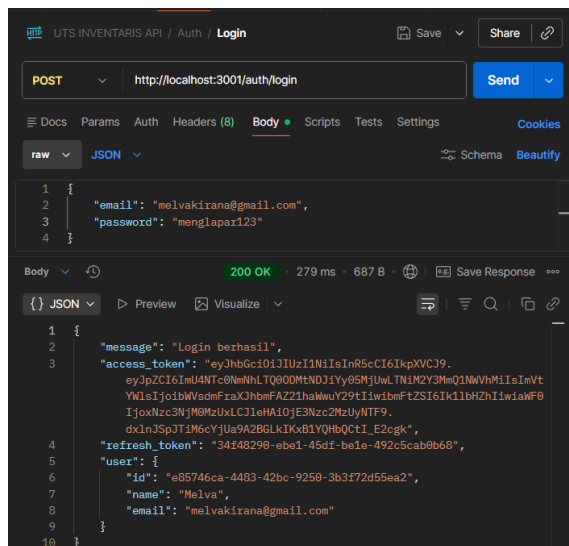
Sistem Inventaris Toko/Gudang berhasil dibangun menggunakan arsitektur microservice dengan 3 service independen dan 1 API Gateway. Sistem ini memenuhi seluruh poin yang disyaratkan meliputi arsitektur microservice dengan database terpisah, REST API dengan PHP CodeIgniter 4 menggunakan pola MVC, autentikasi JWT dengan access token dan refresh token, otorisasi GitHub OAuth 2.0 dengan Authorization Code Flow, serta API Gateway sebagai single entry point dengan JWT validation dan rate limiting. Komunikasi inter-service berhasil diimplementasikan antara report-service dan inventory-service menggunakan HTTP request. Seluruh endpoint telah diuji dan berjalan sesuai dengan yang diharapkan.

H. Bukti Pengujian

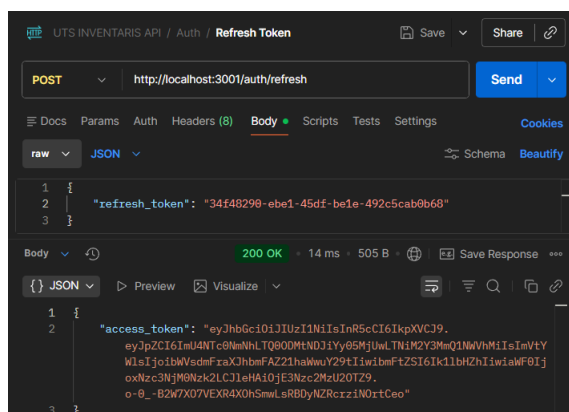
1. POST /auth/register — Registrasi user baru



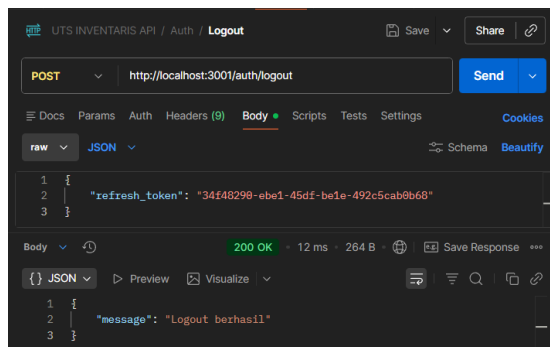
2. POST /auth/login — Login dengan email & password



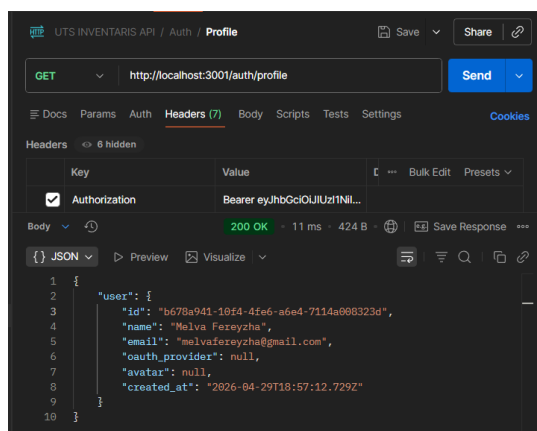
3. POST /auth/refresh — Refresh access token



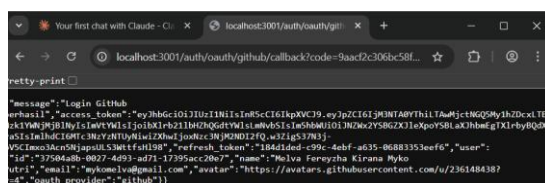
4. POST /auth/logout — Logout & blacklist token



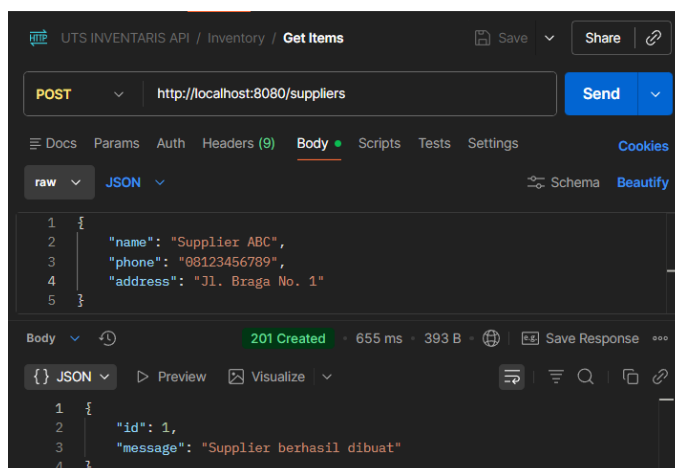
5. GET /auth/profile — Lihat profil user



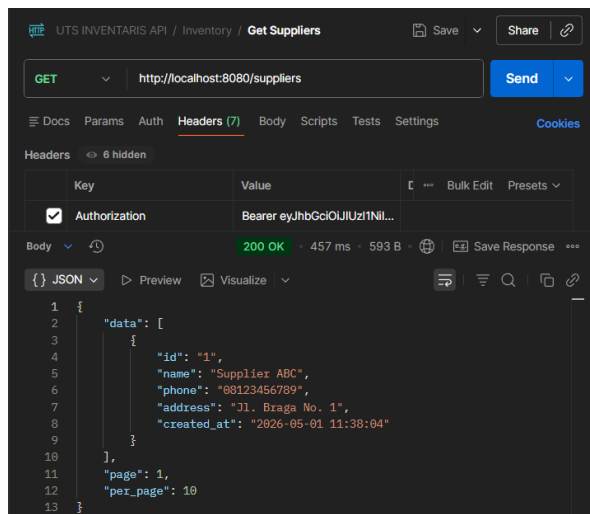
6. GET /auth/oauth/github — GitHub OAuth 2.0



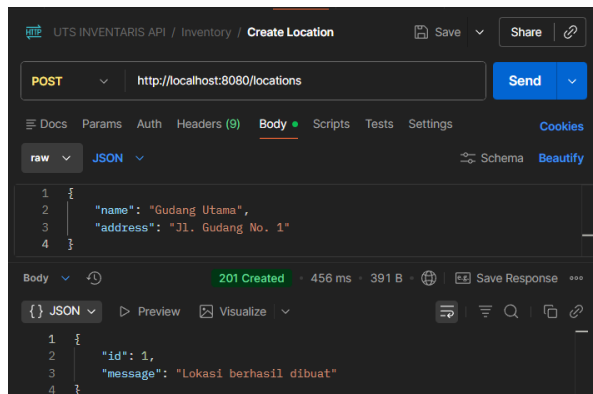
7. POST /suppliers — Tambah supplier baru



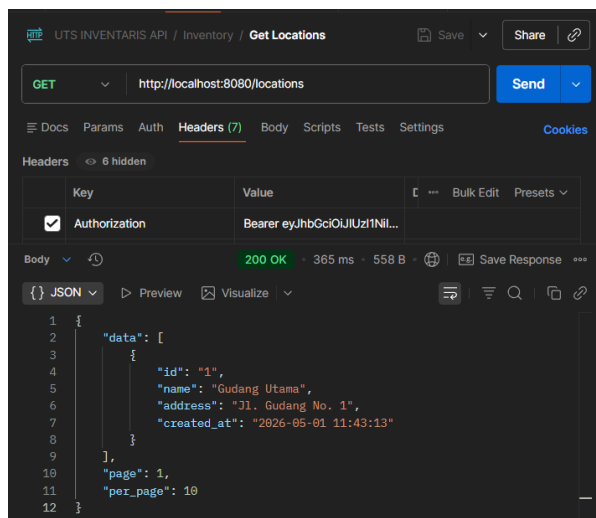
8. GET /suppliers — List semua supplier



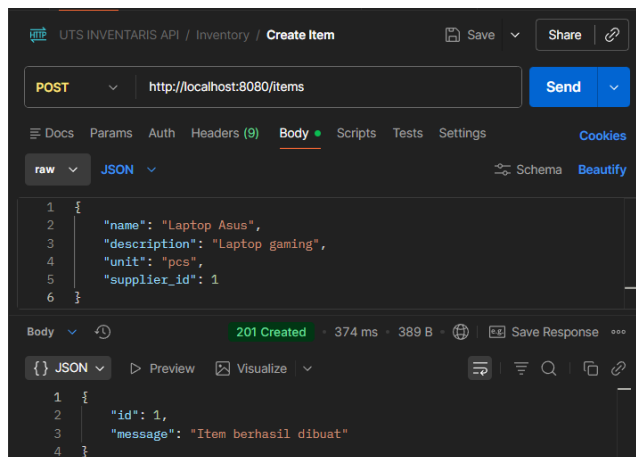
9. POST /locations — Tambah lokasi gudang



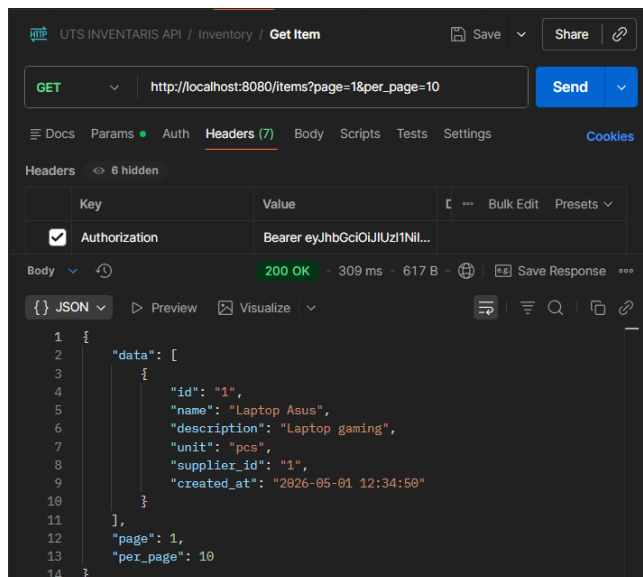
10. GET /locations — List semua lokasi



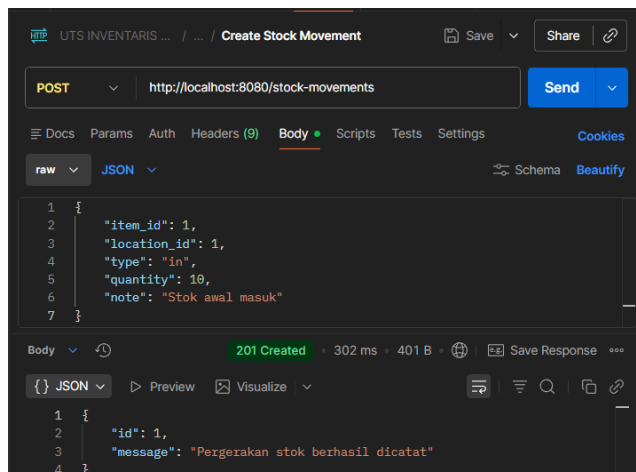
11. POST /items — Tambah barang baru



12. GET /items?page=1&per_page=10 — List barang dengan paging



13. POST /stock-movements — Catat pergerakan stok



14. GET /stock-movements — List pergerakan stok

The screenshot shows a REST client interface for the 'UTS INVENTARIS API'. The request is a GET to 'http://localhost:8080/stock-movements' with a Bearer token in the Authorization header. The response is a 200 OK status with a 403 ms response time and 634 B of data. The JSON response is as follows:

```
1 {
2   "data": [
3     {
4       "id": "1",
5       "item_id": "1",
6       "location_id": "1",
7       "type": "in",
8       "quantity": "10",
9       "note": "Stok awal masuk",
10      "created_at": "2026-05-01 12:37:27"
11    }
12  ],
13  "page": 1,
14  "per_page": 10
15 }
```

15. GET /reports/stock — Laporan summary stok

The screenshot shows a REST client interface for the 'UTS INVENTARIS API'. The request is a GET to 'http://localhost:3003/reports/stock' with a Bearer token in the Authorization header. The response is a 200 OK status with a 450 ms response time and 463 B of data. The JSON response is as follows:

```
1 {
2   "summary": {
3     "total_in": "010",
4     "total_out": 0,
5     "total_movements": 1
6   },
7   "data": [
8     {
9       "id": "1",
10      "item_id": "1",
11      "location_id": "1",
12      "type": "in",
13      "quantity": "10",
14      "note": "Stok awal masuk",
15      "created_at": "2026-05-01 12:37:27"
16    }
17  ],
18  "page": 1,
19  "per_page": 10
20 }
```